

Road Segmentation from Satellite Images with Stacked Neural Networks

Lucas Erlacher, Elise Cho, Benedikt Schesch, Jonas Spieler

Group: Benedikt

Department of Computer Science

ETH Zurich, Switzerland

Abstract—Image segmentation plays an important role in numerous fields, including navigation, medical imaging, and image editing. This report focuses on accurately extracting roads from satellite imagery by employing deep learning techniques. We investigate the use of convolutional neural networks (CNNs) for this task, specifically U-Nets and DeepLabv3+ models. Our work investigates methods for improvement, such as data supplementation and augmentation, different loss functions, and stacking architectures. Our findings indicate that stacked architectures, in particular stacked DeepLabv3+, outperform other models in a variety of quality criteria.

I Introduction

Accurately identifying spatial characteristics in image segmentation is crucial for various applications, including navigation, medical image analysis, and image editing. Precise segmentation is especially vital for autonomous driving and everyday route planning. This study focuses on outlining road positions and proportions in aerial images using modern deep learning techniques. Convolutions are essential for extracting relevant features and capturing spatial patterns, enabling accurate road identification even in challenging situations, like occlusions and similar color distributions between road and non-road segments [1].

A solution to image segmentation that takes advantage of convolutions is to use U-Nets, which are popular due to their usage of skip connections to allow for more precise localization [2]. While standard U-Nets are generally expected to perform well on the segmentation task, modifications such as UNet++ [3] have been suggested in order to properly encapsulate image details in the masks, and thus increase their corresponding accuracy. As such, we also considered improvements to the U-Net model to better suit it for our task.

DeepLabv3+ is a state-of-the-art convolution-based architecture that produces precise and semantically meaningful image segmentation [4]. Due to its ability to better preserve finer details and handle multi-scale objects, through its use of atrous convolutions, we considered this architecture as well as an alternative solution to our problem of road segmentation.

Our work explores the following strategies for improvement:

Data Supplementation. Generating further data samples to train models and increase robustness by scraping

websites like Bing Maps [5], Google Maps [6] and OpenStreetMap [7].

Data Augmentation. Similarly to above, increasing model robustness by applying transformations to the provided dataset.

Loss. We compare results from training with dice loss [8] and binary cross entropy loss.

Architectures. We compare the results of using standard U-Nets versus the cutting-edge DeepLabv3+ architecture.

Stacking Architectures. Comparing the results of baseline models to those of greater complexity, such as stacked U-Net and stacked DeepLabv3+.

The effectiveness of these results are compared in a Kaggle competition hosted by ETH Zurich [9].

II Data

The provided training dataset for this problem consists of 144 RGB images,¹ with a resolution of 400×400 . Each training image has a respective ground truth mask, differentiating the roads and background by assigning labels to the corresponding pixels ($p \in \{0, 1\}$).

Deep learning methods are known to require substantial quantities of training samples for optimal model performance [10]. Therefore, creating a more abundant training dataset was a preliminary goal for this project.

II-A Data Supplementation I: Bing

With many sources available online, we first started with Bing Maps [5] that offered the best open accessible satellite images without any annotations.

In order to get additional training samples, we took a list of coordinates of 43'000 cities around the world (basic World Cities Database) [11] and sampled each coordinate multiple times with a varying offset. This way we got ≈ 4.3 million locations to fetch our data samples from.

For our target masks, we took use of the OpenStreetMaps API [7] to fetch an SVG image within a given bounding box. By extracting the road segments and converting them to a black and white image, we got our target images. A sample from the Bing dataset is shown in Figure 1.

¹Aerial images obtained from Google Maps.



Figure 1. Random grid sampled from our generated Bing dataset including an aerial image and its corresponding mask. Note that not all roads are properly labeled in the masks.

II-B Data Supplementation II: Google

In our evaluation of the Bing dataset, we observed a degree of inaccuracy pertaining to the road width. This, in our assessment, may serve as a crucial impediment to attaining high accuracy. Further scrutiny revealed imperfections within the original supplied training data, with instances of missing road and path details, presumably due to their unofficial status. A comparable pattern was discernible within the Bing dataset, albeit with a differing bias. This, along with the knowledge that the original images were taken from Google Maps² lead us to conjecture that the source of the supplied dataset might have used additional information from Google Maps.

Motivated by these findings, we worked on constructing a precise dataset derived from Google Maps, envisioned to eventually use when fine-tuning our models. Given the limited size of the original dataset — only comprising 144 images — this seemed an optimal approach.

Moreover, it was notable that the original training data was predominantly focused on cities within the United States, with an over-representation of the West Coast region. To use this geographical imbalance we selected to sample 5'000 images from Los Angeles, San Francisco, Sacramento, Portland, Salt Lake City, San Jose, and Seattle.

Illustrative samples derived from the Google Maps dataset are depicted in Figure 2.

II-C Data Augmentation

Although the original provided dataset is quite limited in size, it is preferable to include it in our training pipeline, as it is expected that the ground truth masks would be more representative of the masks in the test dataset. In other words, the train dataset is sampled from the same underlying data distribution P_{data} as the test dataset.

$$X_{train}, X_{test} \sim P_{data}$$

²Noted in the original problem description provided to us.

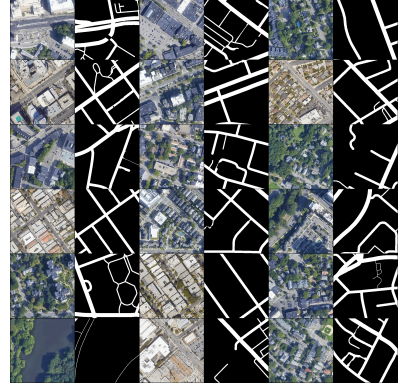


Figure 2. Random grid sampled from our generated Google dataset.

To be able to use the small dataset, we enhanced it by applying varying augmentations and used it to finetune our model. The details and types of augmentations for this process are further outlined in Section III-F.

III Models and Methods

III-A Baseline 1: Linear Model

We initially considered a rudimentary model to see if there are any simple patterns in the original dataset. We did so by implementing a simple linear model consisting of a 1×1 convolution, performing an element-wise linear combination, to collapse the channels of the input image into one final channel for the output mask.

III-B Baseline 2 and 3: Standard U-Net

The next two baselines relied first on a basic U-Net architecture. We implemented our U-Net with an input block, four encoder and four decoder blocks, and an output block. An example of this structure can be seen in U-Net1 in Figure 3. The weights were randomly initialized.

For our first baseline, we first trained the U-Net on our generated Bing dataset. After this preliminary training, we then fine-tuned the model on the augmented set of the original 144 images provided to us. Our second baseline had a similar method, except that we first trained the U-Net on the generated Google data, instead of Bing.

1) Input and Output Blocks

The input block has two 3×3 convolutions with padding and dilation of 2. Batch normalization and Rectified Linear Unit (ReLU) activation are applied after each convolution.

The output block consists of a single 1×1 convolution followed by a Sigmoid activation function.

2) Encoder Block

Each encoder block starts with a 2×2 max pooling layer (stride 2), followed by two 3×3 convolutions (dilation and padding of 2). Batch normalization and ReLU activation are applied after each convolution. The output is used as input for the next block and as a skip connection to corresponding decoder blocks.

3) Decoder Block

For the decoder blocks, we use a 2×2 transposed convolution (stride 2) on the previous block's input. This output is concatenated with the input from the skip connection. Two convolutions with batch normalization and ReLU activation, similar to the encoder blocks, are then applied.

We utilized dilated convolutions to enhance U-Net's performance on small and thin roads by increasing the receptive field.

III-C Stacked U-Net

Upon analyzing the single U-Net outputs, we observed predominantly sparse weight matrices and blurry predictions (visible in the left two prediction columns of Figure 4). Consequently, we made two improvements: First, we reduced the number of parameters in U-Net layers by a factor of 8. Second, we introduced an additional U-Net to refine the blurry predictions from the initial network.

We then implemented a model similar to that of a DoubleU-Net, proposed in [12], where two U-Net architectures are used in sequence. Unlike this example, which has skip connections from the encoder in the first network to the decoder in the second, our proposed architecture simply stacks the initial inputted image data to the output of the first U-Net (U-Net1 in Figure 3) before feeding it into the second U-Net (U-Net2 in Figure 3).

The two U-Nets follow similar internal structures to the basic one outlined in section III-B. The input block for U-Net1 takes in image data with three channels (for RGB values), whereas the input block for U-Net2 takes in four channels, three for the original input image and one for the output of U-Net1. The output of U-Net2 is our prediction of the ground truth mask.

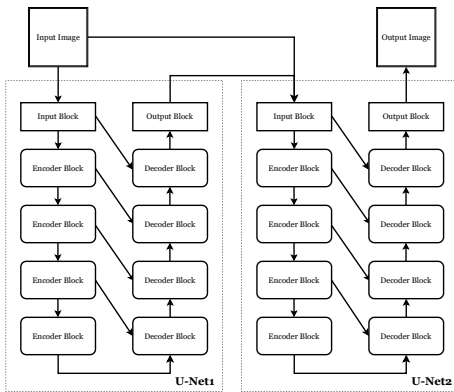


Figure 3. Block diagram of the Stacked U-Net architecture.

During training, the weights were randomly initialized for the Stacked U-Net. We then trained the whole model on the generated Bing dataset, before fine-tuning on the original provided set, much like the process for the single U-Net model.

III-D DeepLabv3+

We followed the DeepLabv3+ architecture described in [4] as another model with the hope of improvements. We initialized with pre-trained weights from the Cityscapes dataset [13] and used a ResNet-101 [14] as the backbone model.

The DeepLabv3+, much like the U-Net, has an encoder-decoder structure, with skip connections from the encoder to the decoder. Similarly to our U-Net encoder implementation, the DeepLabv3+ uses dilated convolutions (atrous convolutions in the ResNet-101) to allow for a greater receptive field. Additionally, after the ResNet-101, DeepLabv3+ uses the Atrous Spatial Pyramid Pooling Module (ASPP) [15], which allows sampling feature maps at varying dilation rates to further capture multi-scale context in the input image. As a result, it may be able to better identify smaller details, which in this case would consist of thin or partially obscured roads in the aerial images.

The skip connections feed the output from the ResNet-101 (that is within the encoder) and the final output from the encoder itself into the decoder.

During training, this model was first pre-trained on the Bing data, before being further trained on the Google dataset, using the weights from the previous training. Afterwards, it was also fine-tuned on the original 144 images.

III-E Stacked DeepLabv3+

This model follows the same stacking format as in the Stacked U-Net, only we replace the U-Nets with DeepLabv3+ architecture. As the DeepLabv3+ outperforms a simple U-Net, and we can see additional improvements when using a U-Net by stacking two U-Nets (Section IV), we decided to combine the ideas of the two approaches to see if further improvements could be observed.

The weights for the first DeepLabv3+ in the stack were taken from the trained DeepLabv3+ in the previous section. The weights for the second one in the stack were randomly initialized. The full model was trained on the Google dataset before being fine-tuned on the limited original dataset.

III-F Data Augmentation

1) Training Dataset

We applied a random affine transformation to each sample pair (of input image and ground truth mask), as well as randomized color jitter. We also randomly applied horizontal and vertical flips to each sample pair.

2) Testing Dataset

To enhance our model's performance, we employed test time augmentation [16]. This technique involves using multiple transformed versions of the input image, passing them through the model, and combining the predictions into a final output. The augmentations applied to our input images included horizontal and vertical flips, as well as 90° , 180° ,

and 270° rotations. To aggregate the predictions, we took the average over all the results.

III-G Training

We experimented with two loss functions during model training: binary cross-entropy (BCE) loss and dice loss. Dice loss was chosen for its ability to handle class imbalance effectively [8], which we hoped would improve the detection of smaller, less noticeable roads.

We minimized our loss to train the models using an AdamW optimizer, initialized with a learning rate of $1 \cdot 10^{-3}$ and a weight decay coefficient of $1 \cdot 10^{-2}$ (default parameters). We reduce the learning rate by a factor of 0.1 whenever no change greater than $1 \cdot 10^{-4}$ is observed for 5 epochs.

For the three baseline models, we used a batch size of 64. For the Stacked U-Net and normal DeepLabv3+ model, we used a batch size of 32. For the Stacked DeepLabv3+, we use a batch size of 16.

We implemented our models in PyTorch. Training was completed on a RTX 3090 and took approximately 2 days for the U-Net and DeepLabv3+ models.

In total, we had three datasets to train on; the original data provided (used for evaluation), generated Bing data ($\approx 40'000$ samples), and generated Google data ($\approx 5'000$ samples).

IV Results

The final test set consisted of 144 images with dimensions $3 \times 400 \times 400$. Results are summarized in Table I, including additional metrics from our evaluation set. The abbreviation “Stkd.” denotes the “Stacked” model, and unless stated otherwise, models were trained using BCE Loss. For a visual comparison of the models, refer to Figure 4.

Methods	Results on Data				
	Public Score	Accuracy	F1 Score	Precision	Recall
PixelWiseLinear	0.24448	0.177 ± 0.015	0.300 ± 0.022	0.177 ± 0.015	1.000 ± 0.000
Bing U-Net	0.87568	0.928 ± 0.002	0.783 ± 0.019	0.840 ± 0.008	0.734 ± 0.028
Google U-Net	0.89804	0.933 ± 0.002	0.799 ± 0.021	0.850 ± 0.011	0.753 ± 0.028
Stkd. U-Net	0.91180	0.941 ± 0.005	0.826 ± 0.015	0.859 ± 0.016	0.796 ± 0.022
DeepLabv3+	0.92388	0.918 ± 0.008	0.726 ± 0.035	0.886 ± 0.012	0.616 ± 0.046
Stkd. DeepLabv3+	0.93282	0.958 $\pm$ 0.006	0.879 $\pm$ 0.016	0.900 $\pm$ 0.011	0.860 $\pm$ 0.024
Stkd. DeepLabv3+ w/ Dice Loss	0.92613	0.954 ± 0.006	0.866 ± 0.016	0.898 ± 0.013	0.837 ± 0.026

Table I
MODEL RESULTS FROM TRAINING.

While we had hopes that we would be able to detect patterns in the aerial images through our 1×1 convolution model, we quickly realized that the solution would be more complex. We were able to confirm our conjecture that the generated Google dataset would be a better representation of the test data, as observed by the difference in our U-Net baselines in Table I. Also considering that the F1 score, as shown in Figure 5, converged faster during the training of the U-Net on Google dataset vs on the Bing dataset with

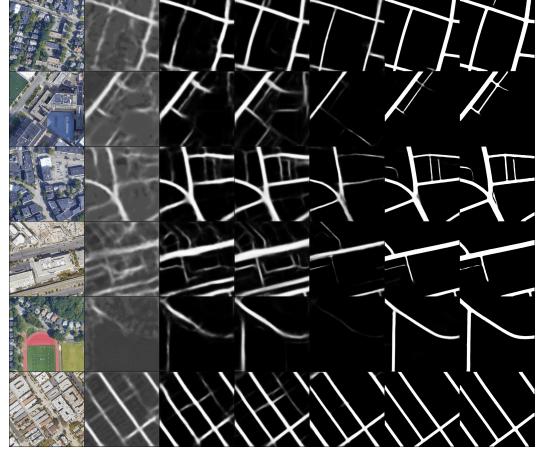


Figure 4. Visualization of the model outputs on the test dataset from the Kaggle competition (from left to right: satellite image, U-Net (Bing data), U-Net (Google data), Stkd. U-Net, DeepLabv3+, Stkd. DeepLabv3+, Stkd. DeepLabv3+ w/ Dice Loss).

all other parameters identical, this also supports our claim (even though the F1 score was smaller, the Google U-Net performed better after fine-tuning it on the train dataset).

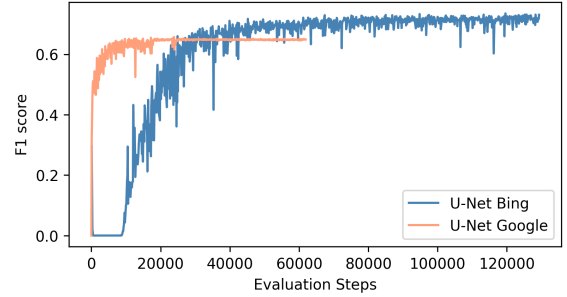


Figure 5. F1 score convergence during training the U-Net on Bing data vs on Google Data

We observed a noticeable improvement from our baselines with our stacked and DeepLabv3+ models. However, we see that the DeepLabv3+ architecture, while performing better than the Stacked U-Net and producing crisper roads (see Figure 4), struggled more with identifying thin and obscured roads. Unexpectedly, we noted reduced accuracy when training on Dice loss versus training on BCE loss.

V Summary

In this work, we showcased the benefits of combining one of two architectures, U-Nets and DeepLabv3+, into a stacked architecture for aerial image segmentation. We were able to show that by stacking an otherwise simple model (U-Net) and re-introducing the original image data, our model achieved a boost in performance. Additionally, we demonstrated that integrating the state-of-the-art DeepLabv3+ led to further improvements.

References

- [1] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- [2] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. *CoRR*, abs/1505.04597, 2015. URL <http://arxiv.org/abs/1505.04597>.
- [3] Zongwei Zhou, Md Mahfuzur Rahman Siddiquee, Nima Tajbakhsh, and Jianming Liang. Unet++: A nested u-net architecture for medical image segmentation. *CoRR*, abs/1807.10165, 2018. URL <http://arxiv.org/abs/1807.10165>.
- [4] Liang-Chieh Chen, Yukun Zhu, George Papandreou, Florian Schroff, and Hartwig Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation. *CoRR*, abs/1802.02611, 2018. URL <http://arxiv.org/abs/1802.02611>.
- [5] Bing maps. <https://www.bing.com/maps/>. Accessed: 2023-07-16.
- [6] Google maps. <https://www.google.com/maps>. Accessed: 2023-07-16.
- [7] Openstreetmaps. <https://www.openstreetmap.org/>. Accessed: 2023-07-16.
- [8] Fausto Milletari, Nassir Navab, and Seyed-Ahmad Ahmadi. V-net: Fully convolutional neural networks for volumetric medical image segmentation. In *2016 Fourth International Conference on 3D Vision (3DV)*, pages 565–571, 2016. doi: 10.1109/3DV.2016.79.
- [9] CIL Lecture. Ethz cil road segmentation 2023, 2023. URL <https://kaggle.com/competitions/ethz-cil-road-segmentation-2023>.
- [10] Chen Sun, Abhinav Shrivastava, Saurabh Singh, and Abhinav Gupta. Revisiting unreasonable effectiveness of data in deep learning era, 2017.
- [11] World cities database. <https://simplemaps.com/data/world-cities>. Accessed: 2023-07-16.
- [12] Debesh Jha, Michael A. Riegler, Dag Johansen, Pål Halvorsen, and Håvard D. Johansen. Doubleu-net: A deep convolutional neural network for medical image segmentation, 2020.
- [13] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The cityscapes dataset for semantic urban scene understanding. *CoRR*, abs/1604.01685, 2016. URL <http://arxiv.org/abs/1604.01685>.
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015. URL <http://arxiv.org/abs/1512.03385>.
- [15] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy, and Alan L. Yuille. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *CoRR*, abs/1606.00915, 2016. URL <http://arxiv.org/abs/1606.00915>.
- [16] Nathan Hubens. Test time augmentation (tta) and how to perform it with keras. <https://towardsdatascience.com/test-time-augmentation-tta-and-how-to-perform-it-with-keras-4ac19b67fb4d>, 2019. Accessed: 2023-07-26.



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every semester paper, Bachelor's thesis, Master's thesis and any other degree paper undertaken during the course of studies, including the respective electronic versions.

Lecturers may also require a declaration of originality for other written papers compiled for their courses.

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor.

Title of work (in block letters):

Road Segmentation from Satellite Images with Stacked Neural Networks

Authored by (in block letters):

For papers written by groups the names of all authors are required.

Name(s):

Erlacher

Cho

Schesch

Spieler

First name(s):

Lucas

Elise

Benedikt

Jonas

With my signature I confirm that

- I have committed none of the forms of plagiarism described in the '[Citation etiquette](#)' information sheet.
- I have documented all methods, data and processes truthfully.
- I have not manipulated any data.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for plagiarism.

Place, date

Zürich, 29. July 2023

Signature(s)

Ben

Jonas

Elise

Lucas

For papers written by groups the names of all authors are required. Their signatures collectively guarantee the entire content of the written paper.